

Movie Collection Ecosystem: Project Overview

A comprehensive blueprint tailored for transitioning .NET experience into modern Full-Stack and Automation ecosystems.

1. Recommended Architecture & Directory Layout

Keep your automated Python scraper and Full-Stack web application in separate modules to keep GitHub Actions workflows clean and highly decoupled.

```
movie-ecosystem/
├── .github/workflows/
│   ├── web-app-ci.yml # CI/CD for React and .NET API
│   └── scraper-ci.yml # CI/CD for Python Scraper
├── backend-api/ # .NET Web API
│   ├── MovieCatalog.API/ # Controllers, Program.cs, AppSettings
│   ├── MovieCatalog.Core/ # Entities (Movie, Watchlist), Interfaces
│   ├── MovieCatalog.Data/ # Postgres DbContext, Migrations
│   └── MovieCatalog.Tests/ # xUnit & Moq Unit Tests
├── frontend-ui/ # React App (Vite + TypeScript)
│   ├── src/
│   │   ├── components/ # Reusable UI (MovieCard, Navbar)
│   │   ├── services/ # API fetch functions
│   │   ├── hooks/ # Custom React hooks (e.g., useMovies)
│   │   ├── package.json
│   │   └── vitest.config.ts # Frontend Unit Testing setup
│   └── python-scraper/ # Python Automation Script
│       ├── src/
│       │   ├── scraper.py # Main scraping logic (BeautifulSoup / Requests)
│       │   └── notifier.py # Discord Webhook / Email alerts
│       └── tests/ # Pytest folder
└── requirements.txt # Python dependencies
```

2. Initial PostgreSQL Schema

Use Entity Framework Core Code-First migrations in your .NET API to set up the following relational database structure in PostgreSQL:

```
-- Movies Table
CREATE TABLE "Movies" (
    "Id" INT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    "Title" VARCHAR(255) NOT NULL,
    "ReleaseYear" INT,
    "Format" VARCHAR(50), -- e.g., '4K Ultra HD', 'Blu-ray', 'Digital',
    "TmdbId" INT UNIQUE, -- Linked with Python Scraper Metadata
    "Rating" INT CHECK ("Rating" BETWEEN 1 AND 5),
    "CreatedAt" TIMESTAMP DEFAULT CURRENT_TIMESTAMP
```

```
);

-- Watchlist Table (For automated price tracking and alert thresholds)
CREATE TABLE "Watchlist" (
  "Id" INT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
  "MovieTitle" VARCHAR(255) NOT NULL,
  "TargetPrice" DECIMAL(5,2),
  "IsNotified" BOOLEAN DEFAULT FALSE
);
```

3. Target Project Core Layout

Project 1: The Movie Catalog & Tracker

- **Backend API:** Developed using .NET Web API (ASP.NET Core). Focus on building clean, RESTful JSON endpoints, global error handling middleware, and structural routing.
- **Frontend Client:** Structured with React using Vite and TypeScript. Leverage decoupled states via Custom Hooks or TanStack Query to manage collection views seamlessly.
- **Unit Testing:** Isolate dependencies with Moq or NSubstitute and assert logic via xUnit to test API endpoints without spinning up live database transactions.
- **GitHub Actions:** Establish automated workflows to restore dependencies, compile projects, and run validation test suites on every pull request or master push.

Project 2: Automated Scraper & Metadata Enrichment

- **Python Scripting:** Build modular scripts using virtual environments (venv) and dependency lists (requirements.txt).
- **Automation:** Utilize Requests to fetch movie data from The Movie Database (TMDb) API and BeautifulSoup or Playwright to sweep physical media storefronts for discounts.
- **Alerting:** Integrate system event webhooks (e.g., Discord or Slack channels) to instantly message you when prices drop below the thresholds tracked in your Postgres DB.
- **Testing & CI:** Formulate structural assertions using Pytest and mock live external HTTP responses with the responses library to keep actions efficient and fast.

4. Sample GitHub Actions CI/CD Template (.github/workflows/web-app-ci.yml)

```
name: Build and Test Full-Stack App
on: [push, pull_request]
jobs:
  backend-ci:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - name: Setup .NET
        uses: actions/setup-dotnet@v4
        with: { dotnet-version: '8.0.x' }
      - name: Restore & Build
        run: |
          dotnet restore backend-api/
          dotnet build backend-api/ --no-restore
      - name: Run Backend Tests
        run: dotnet test backend-api/ --no-build
  frontend-ci:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - name: Setup Node.js
        uses: actions/setup-node@v4
        with: { node-version: '20' }
      - name: Install & Test React
        run: |
          cd frontend-ui
          npm ci
          npm run build
```